Week 4 Assignment

Azure Cloud Fundamentals and Data Pipeline Implementation using ADF

Celebal Excellence Internship Program 2026
Domain: Data Engineering
Submitted by: Kinshu Sachdeva
Date: 14 June 2026

Objective

Understanding basic concepts in Azure cloud platform and developing a complete data pipeline using Azure Storage Account and Azure Data Factory (ADF). The pipeline involves importing CSV files from Blob storage, metadata validation of the file, transferring the file to the desired location, and using rolebased access control (RBAC).

This work is divided into tasks and described sequentially. For each task, the requirements have been mentioned, steps followed, and the outputs captured in screenshots.

Environment Setup

- Resource Group: Celebal_Week_4 (Region: Central India)
- Storage Account: week4blobstorage07 (Standard, LRS)
- Containers: source (input) and destination (output)
- Data Factory: week4Kinshu
- Dataset used: Sample Superstore.csv

Task 1 Explore Azure Portal & Create a Resource Group

Question

Explore the Azure Portal and create a Resource Group that will hold all resources for this assignment.

Steps Followed

1. Visited the Azure portal at portal.azure.com and looked at the dashboard along with the services.
2. Navigated to Resource groups and clicked Create.
3. Set the name to Celebal_Week_4, selected the subscription, and chose Central India as the region.
4. Reviewed and created the resource group, all later resources were placed inside it for easy management.

Output

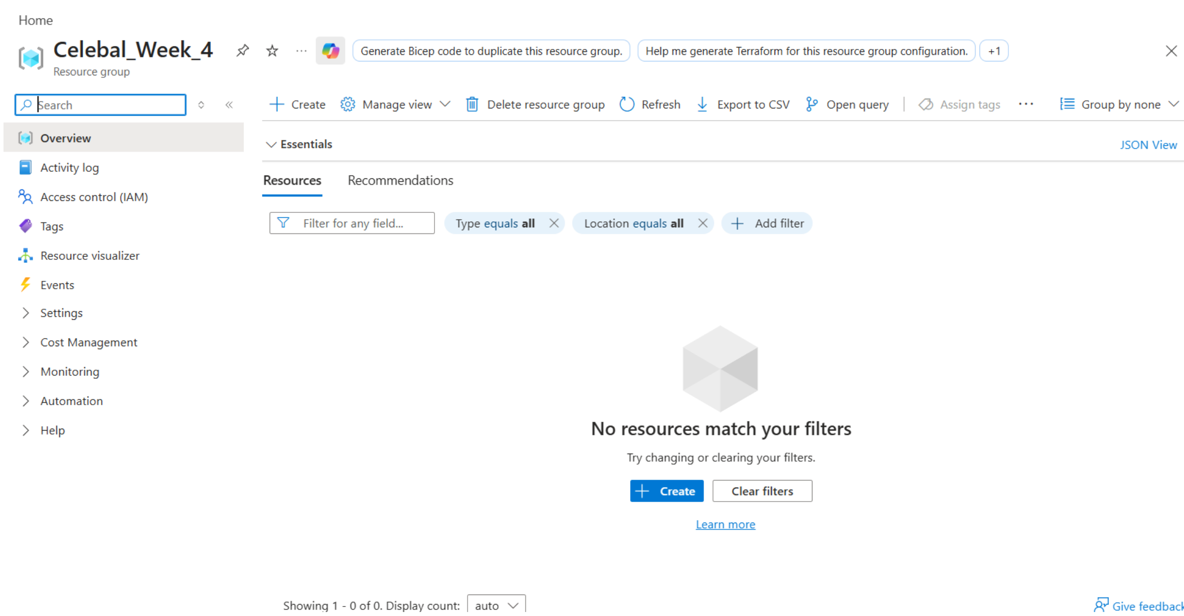

Figure 1 Resource Group 'Celebal_Week_4' created successfully.

Task 2 Storage Setup

Question

Create a Storage Account, create a Blob Container, and upload a CSV file.

Steps Followed

1. Storage Account named week4blobstorage07 (performance: standard, redundancy: locallyredundant storage/LRS) was created in Celebal_Week_4 resource group, where the main service has been set up as Azure Blob Storage.
2. Upon setting up, the storage account was accessed, and under Data storage Containers, two containers were created, which are source (input file), and destination (pipeline output).
3. The file SampleSuperstore.csv was uploaded into the source container.

Output

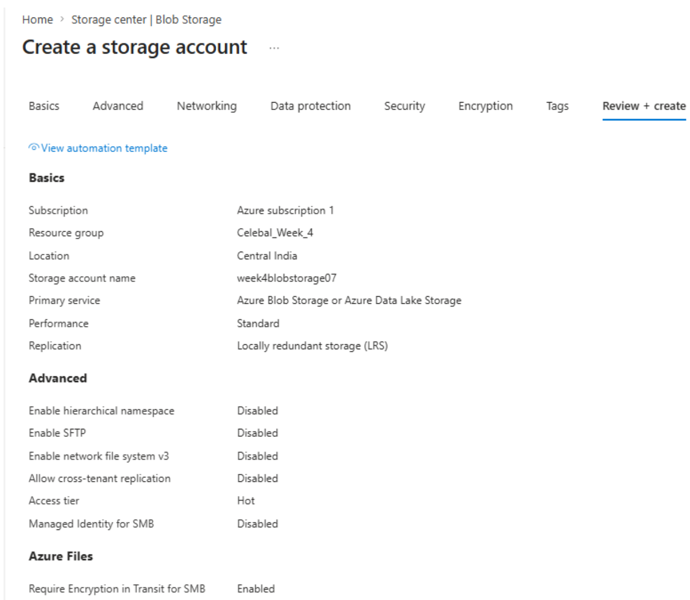

Figure 2 Storage Account configuration (Review + create).

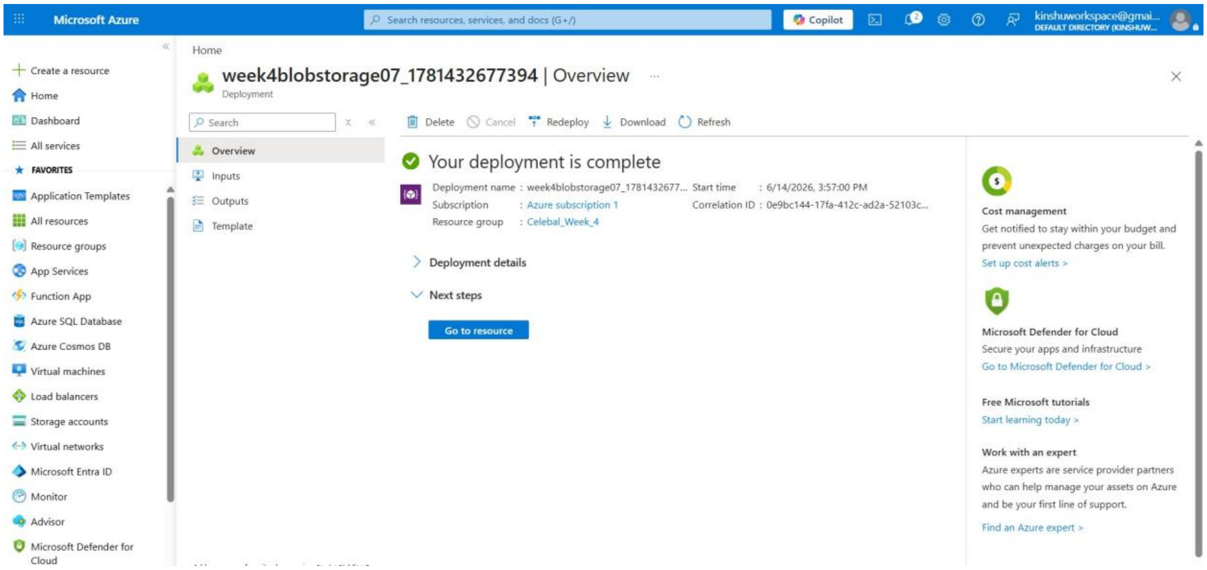

Figure 3 Storage Account 'week4blobstorage07' deployed successfully in resource group Celebal_Week_4.

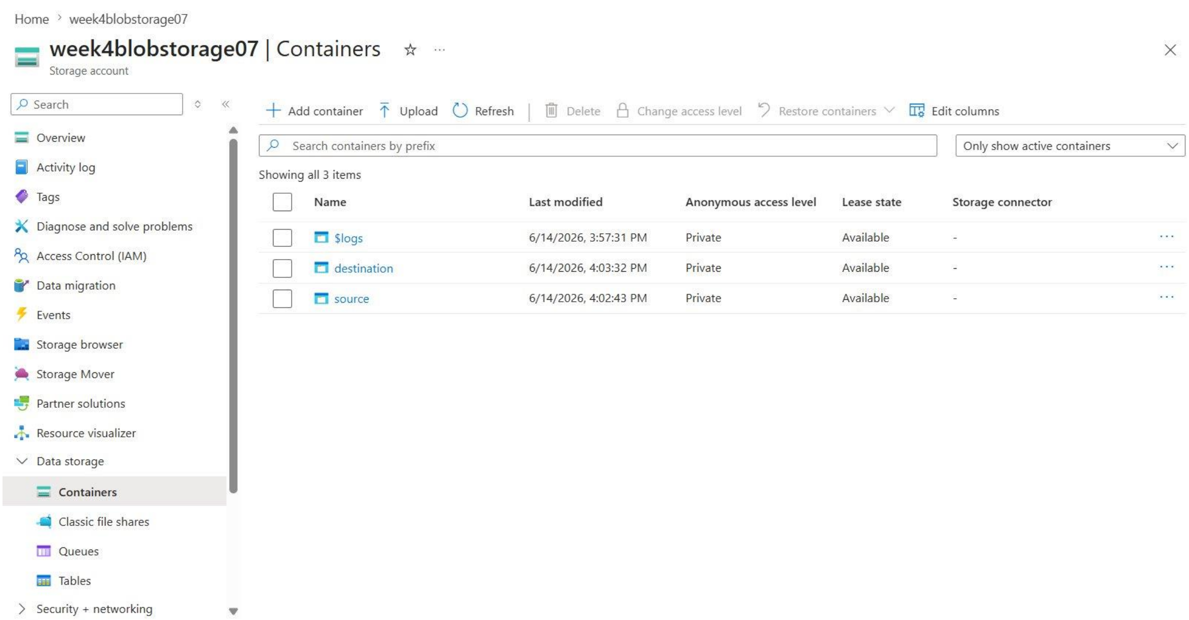

Figure 4 Containers 'source' and 'destination' created in the storage account.

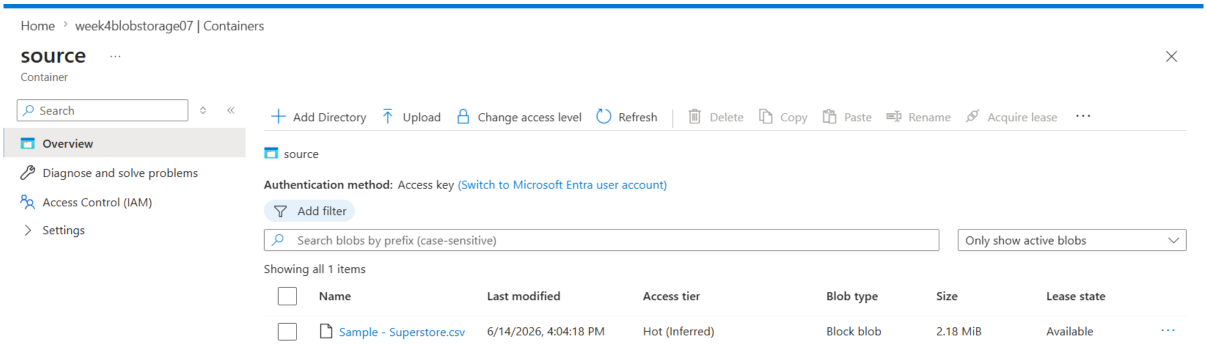

Figure 5 'source' container with the uploaded Sample Superstore.csv file.

Task 3 ADF Basics

Question

Create an Azure Data Factory, explore the ADF UI (Author, Monitor, Manage) create a Linked Service for Blob Storage, create source and destination datasets, and use the Get Metadata activity.

Steps Followed

1. Created an Azure Data Factory named week4Kinshu inside the resource group and launched Azure Data Factory Studio.
2. Explored the three main hubs: Author (build pipelines/datasets), Monitor (view pipeline runs), and Manage (linked services, triggers).
3. In the section Manage Linked Services, Blob Storage linked service(ls_blob_source) is created using account key type, and the connectivity status is checked by executing Test Connection successfully
4. Within the Author node, the source dataset (ds_source_csv) is created based on Sample Superstore.csv while the destination dataset (ds_dest_csv) is created for the destination folder.
5. Added a Get Metadata activity referencing the source dataset and configured its field list to return Item name, Size, and Column count for validation.

Output

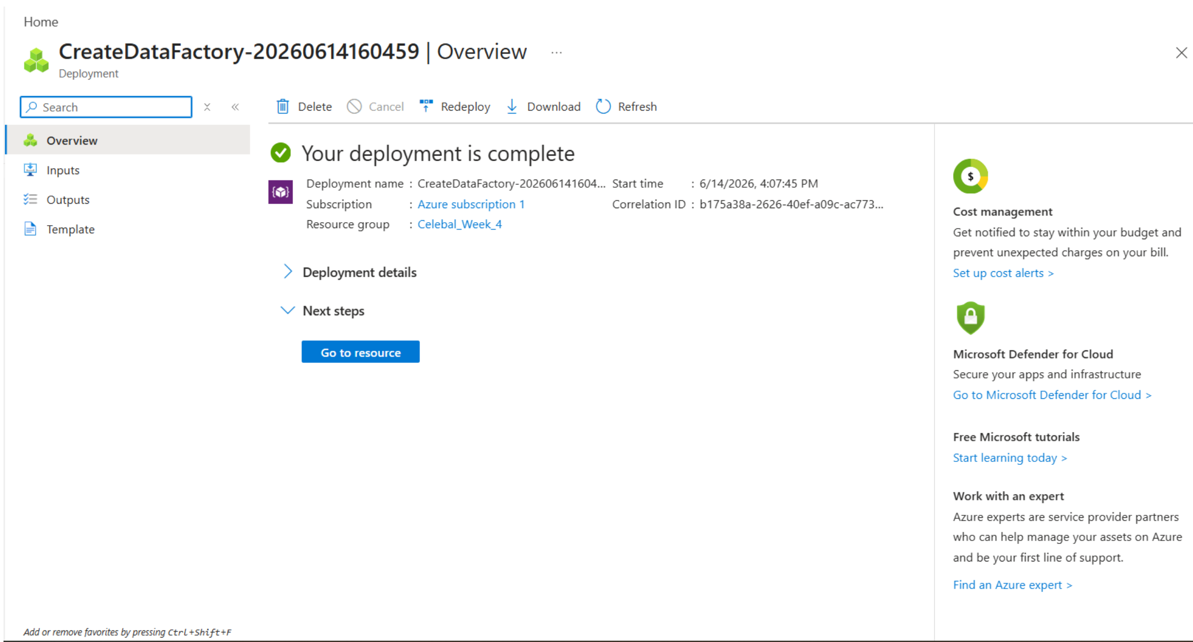

Figure 6 Azure Data Factory deployed successfully.

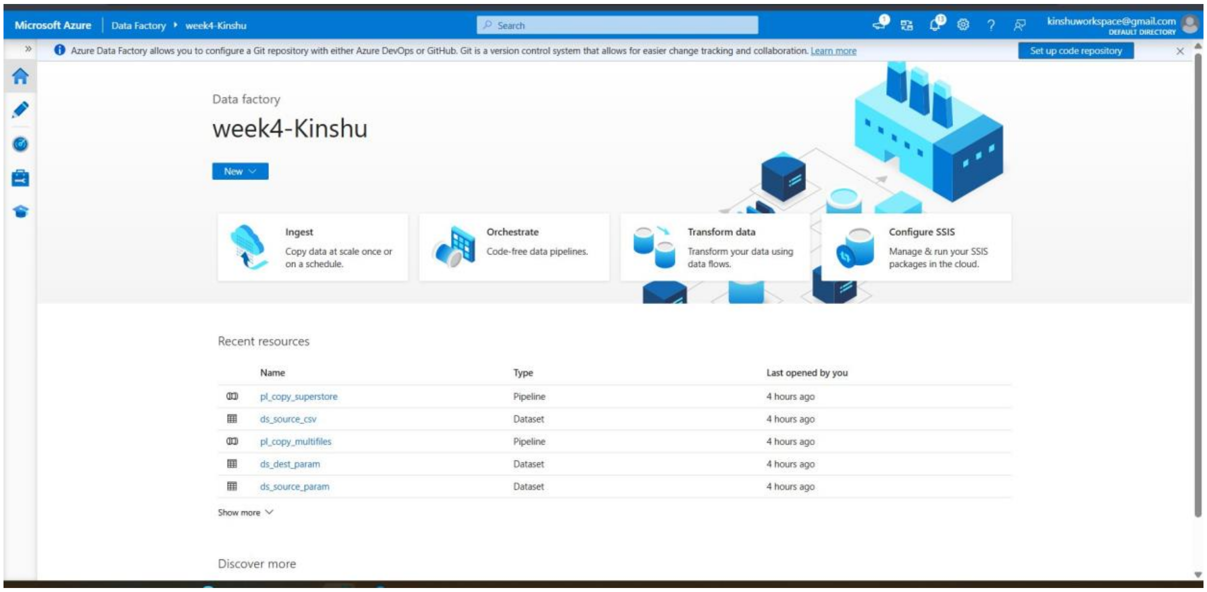

Figure 7 Azure Data Factory Studio (week4Kinshu) showing the Ingest, Orchestrate and Transform options and the Author/Monitor/Manage hubs.

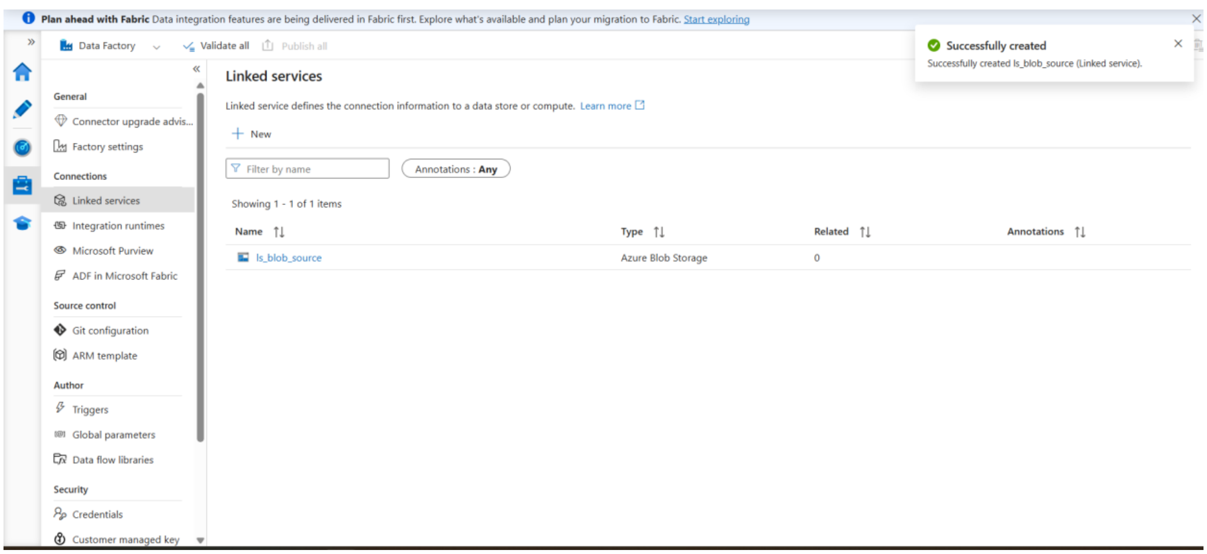

Figure 8 Linked Service 'ls_blob_source' (Azure Blob Storage) created.

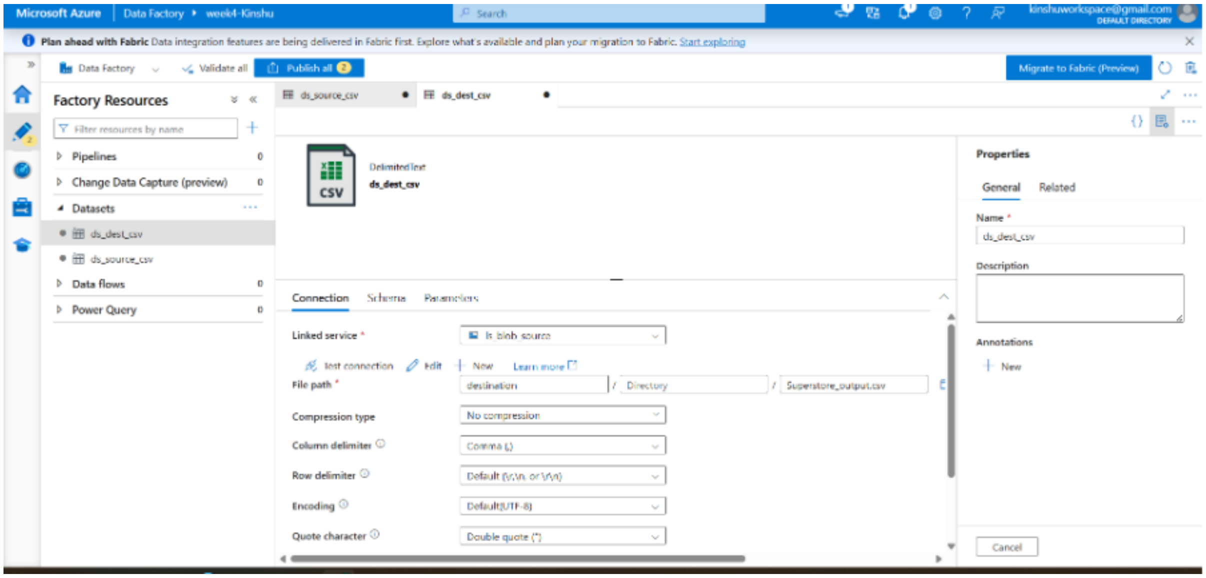

Figure 9 Dataset configuration (DelimitedText / CSV).

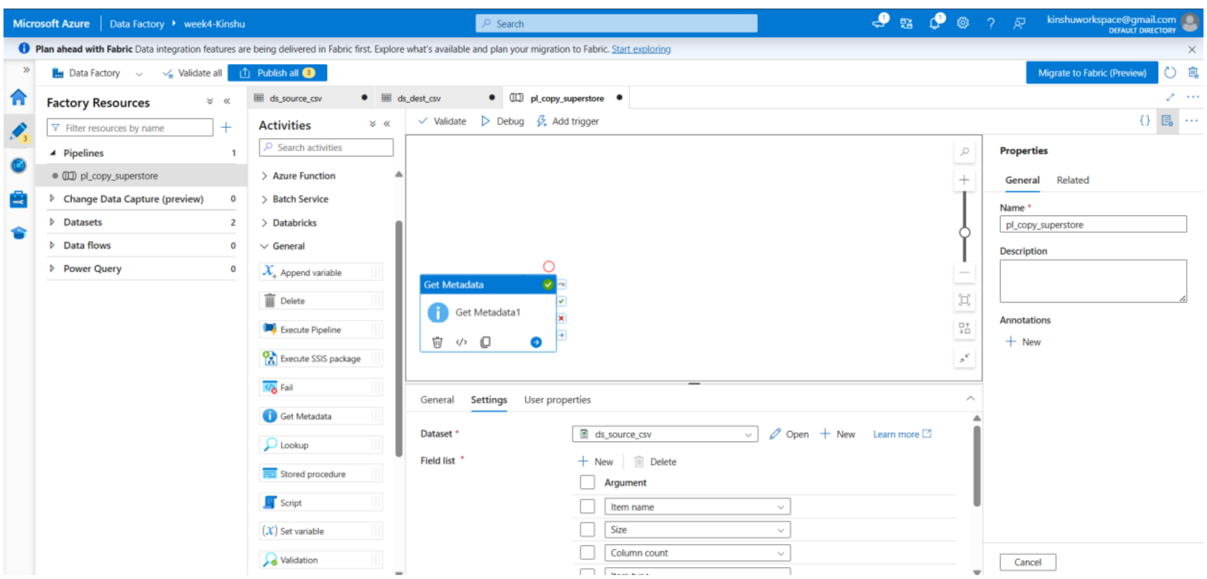

Figure 10 Get Metadata activity with field list (Item name, Size, Column count).

Task 4 Pipeline Development

Question

Create a pipeline using the Copy Data activity, and configure the source and destination.

Steps Followed

1. Created a pipeline named pl_copy_superstore.
2. Added a Get Metadata activity followed by a Copy data activity, connecting the green 'on success' output of Get Metadata to Copy data so the copy runs only after metadata validation succeeds.
3. Configured the Copy data Source to ds_source_csv and the Sink to ds_dest_csv.
4. Validation of the pipeline was done with zero errors found.

Output

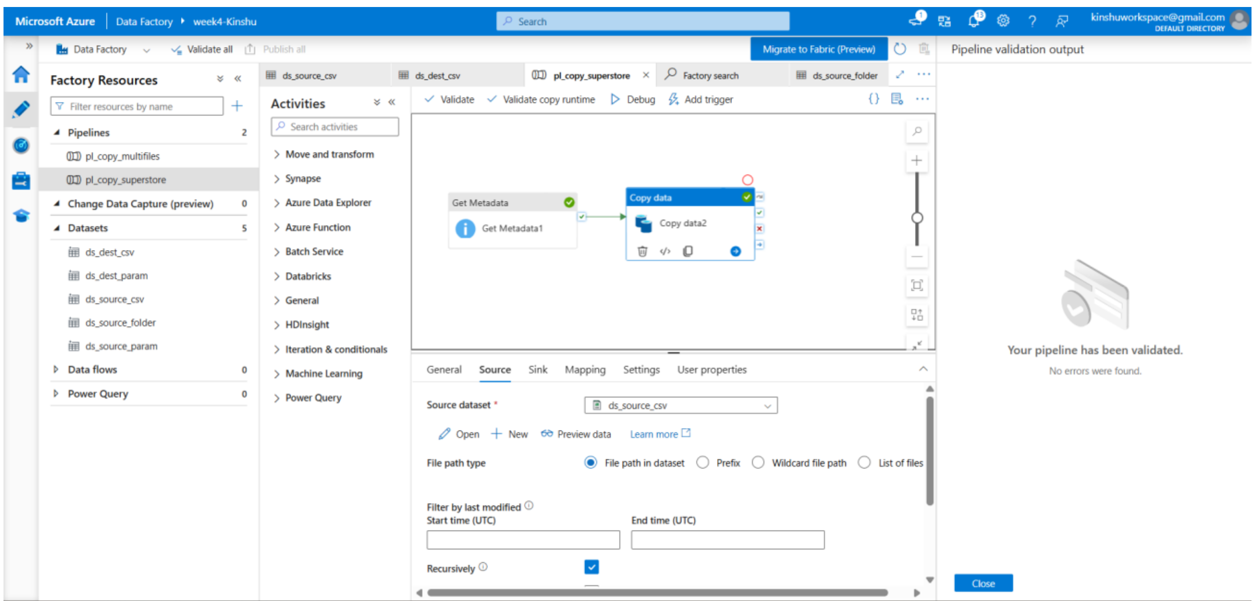

Figure 11 Pipeline design: Get Metadata → Copy data (validated, no errors found).

Task 5 Pipeline Execution

Question

Run the pipeline using Debug / Trigger and confirm successful execution.

Steps Followed

1. Published all changes to the Data Factory service.
2. Ran the pipeline using Debug and monitored the Output tab.
3. It was seen that Get Metadata as well as Copy Data activities both succeeded.

Output

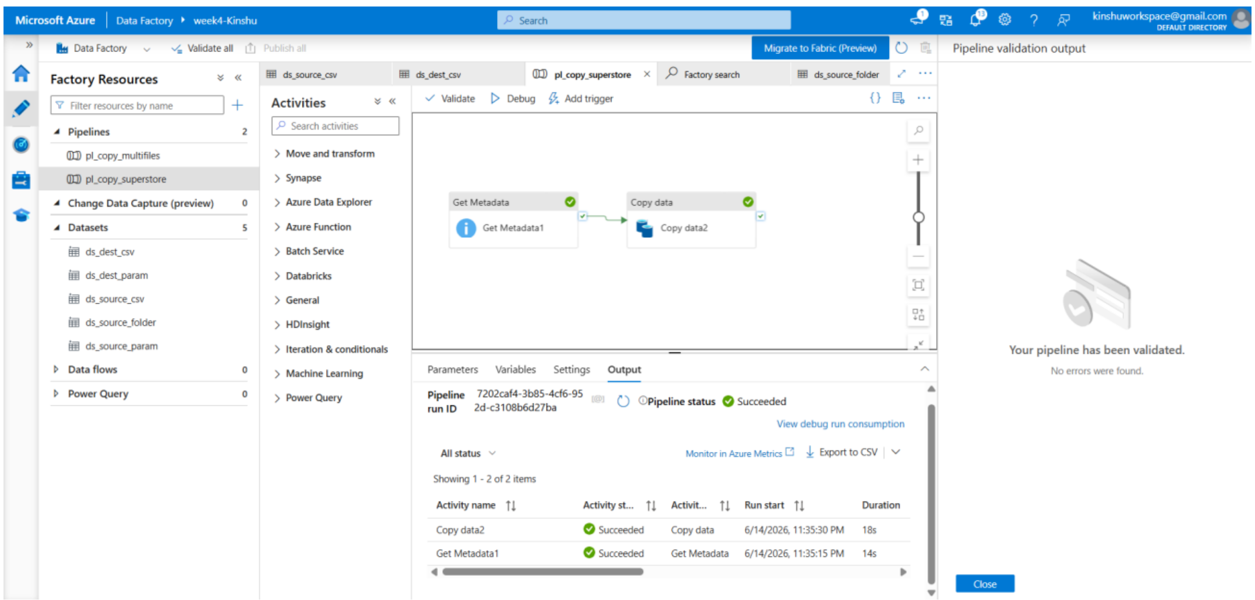

Figure 12 Pipeline executed successfully (Pipeline status: Succeeded).

Task 6 Output Validation

Question

Validate the output of the pipeline by confirming that the source CSV was successfully copied to the destination location with its data and metadata intact.

Steps Followed

1. Opened the destination container in the storage account after the pipeline run completed.
2. Confirmed that the new output file (Superstore_output.csv, 2.18 MiB) was created, proving the data was copied end to end (Blob ADF Blob).
3. The metadata validation process through the Get Metadata activity was confirmed, as the item name, size, and columns were retrieved at runtime.
4. Thus, it is confirmed that the endtoend requirement has been met: Source CSV in Blob Linked service, Dataset, and Pipeline Copy Data with metadata check Creation of the output file at the destination blob.

Output

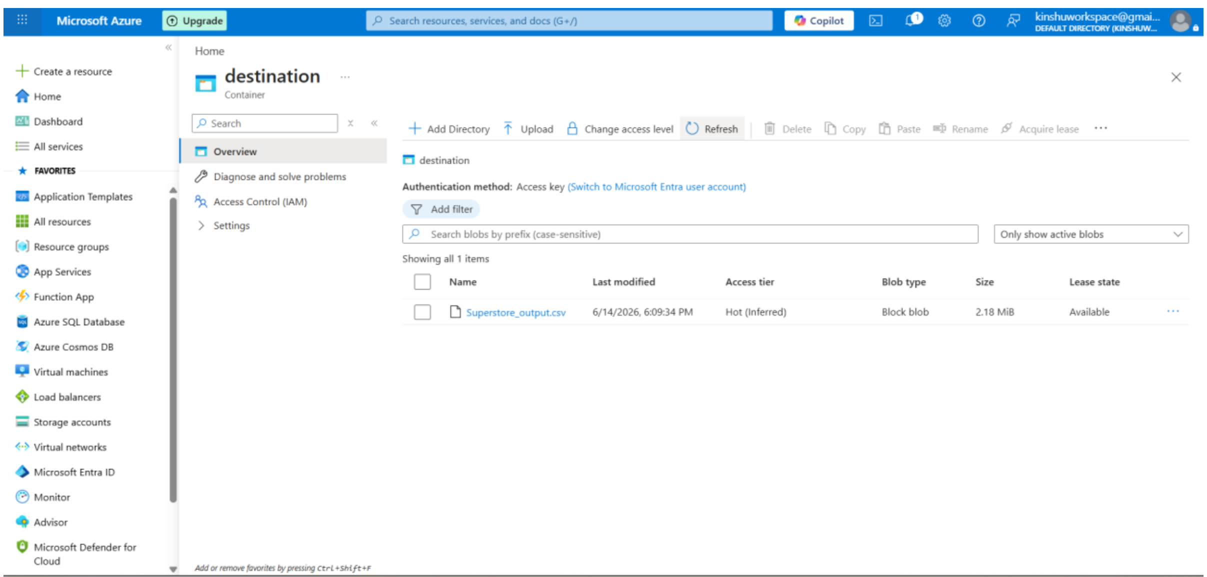

Figure 13 Destination container showing the copied output file (Superstore_output.csv) output validated successfully.

Task 7 IAM Roles

Question

Assign Reader and Contributor roles and provide ADF access to the Storage Account.

Steps Followed

1. Opened the storage account's Access Control (IAM) blade and reviewed Role assignments.
2. The Reader role was given to the user account at the storage account level.
3. Granted the Data Factory's managed identity (week4Kinshu) the Storage Blob Data Contributor role this is the key assignment that lets ADF read/write blobs securely via its managed identity rather than stored keys.
4. Note on Contributor: the account already holds the Owner role (inherited from the subscription). Because Owner is a superset of Contributor, Azure does not allow a redundant Contributor assignment at the same scope, so Owner satisfies and exceeds the Contributor requirement.

Output

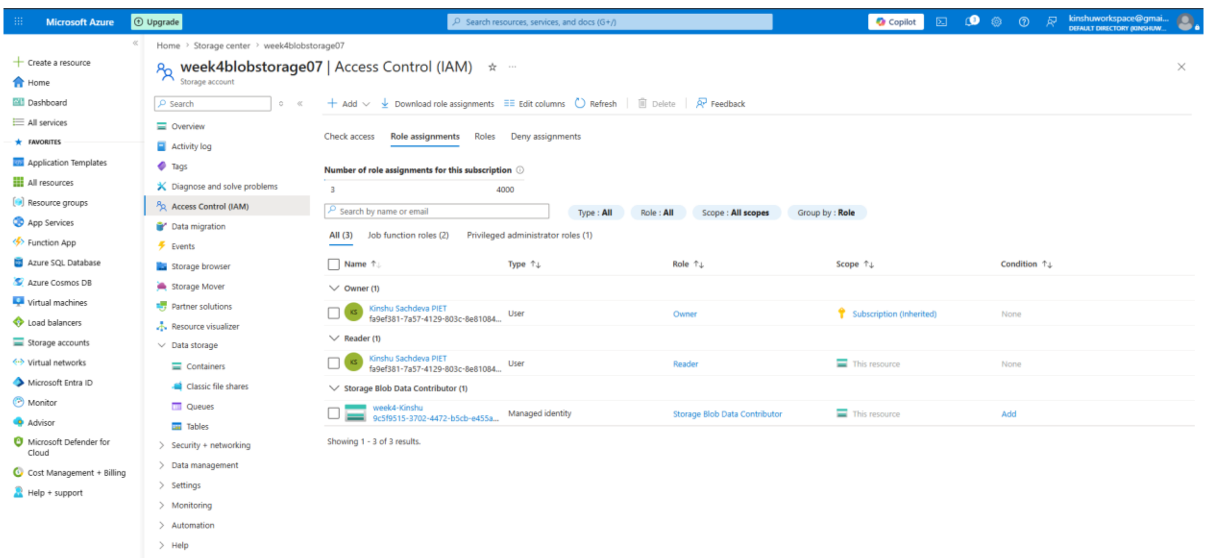

Figure 14 Role assignments: Owner (inherited), Reader, and Storage Blob Data Contributor (ADF managed identity).

Beyond the Requirements

Aside from those tasks required by the assignment, two additional improvements were done to demonstrate data engineering practices in production environment.

1. Dynamic MultiFile Pipeline using ForEach

Rather than hardcoding a single file, another pipeline was created (pl_copy_multifiles) to dynamically scan and copy all files present in the source folder:

- A folder level dataset (ds_source_folder) and a Get Metadata activity return the Child Items (list of all files in the container).
- A ForEach activity loops over that list.
- Parameterized datasets (ds_source_param / ds_dest_param) use @item().name so each iteration reads and writes the correct file.

This makes the pipeline reusable for any number of input files without modification. Three CSV files were placed in the source container and all three were copied in a single run.

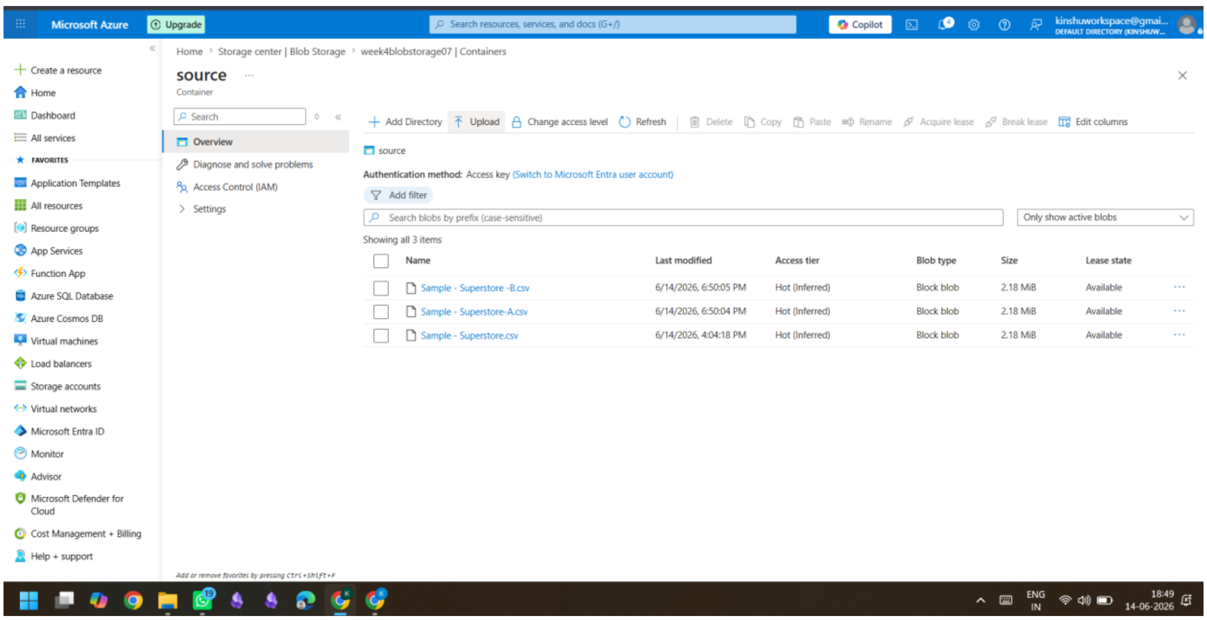

Figure 15 Source container with three CSV files for the ForEach loop.

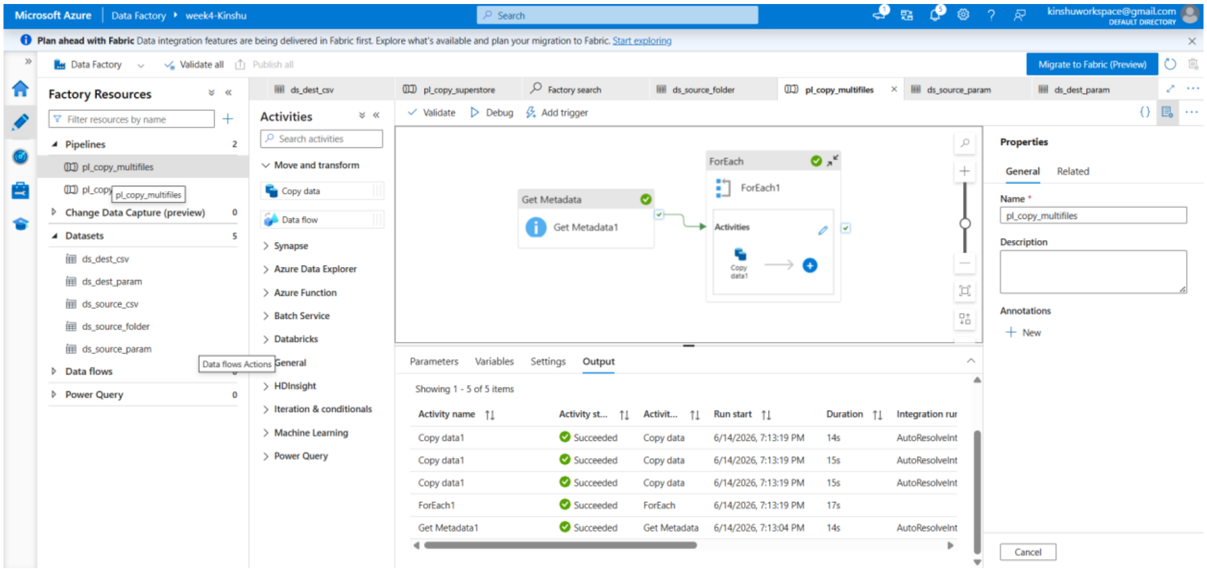

Figure 16 Multifile pipeline: Get Metadata → ForEach → Copy data (Copy ran 3 times, all Succeeded).

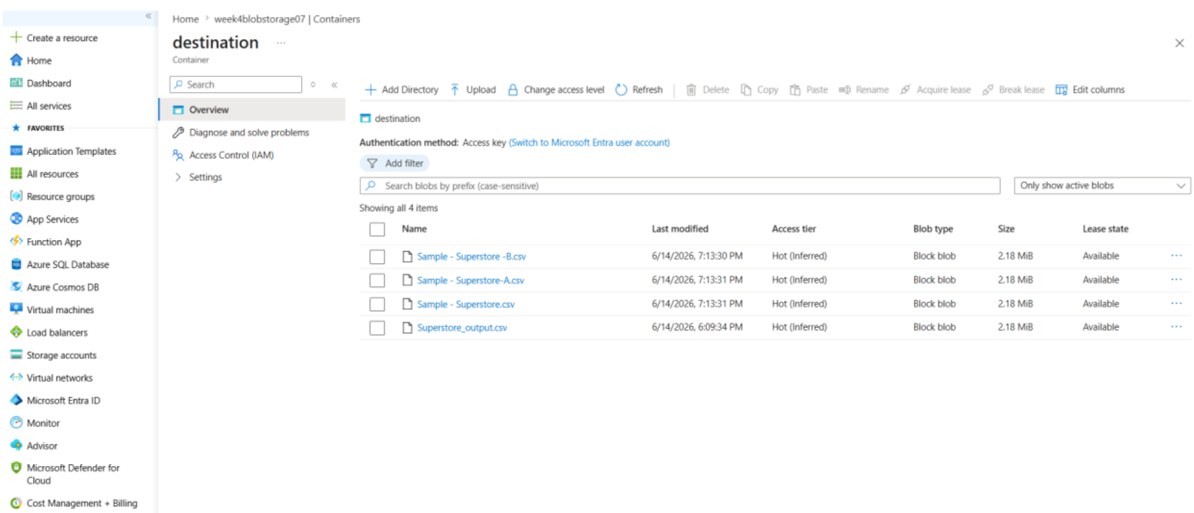

Figure 17 Destination container with all files copied by the ForEach loop.

2. Data Transformation & Fault Tolerance

The Mapping tool available under the Copy activity in the Single File Pipeline design was used to manipulate the incoming data through picking some columns and renaming the 'Customer Name' column to 'Customer'. Data manipulation can be seen from this example in addition to the data transfer.

While column matching faced difficulty with a record having a comma included within its quotes the most prevalent CSV data quality challenge that exists in practiceFault Tolerance was activated using the 'Skip incompatible rows' choice.

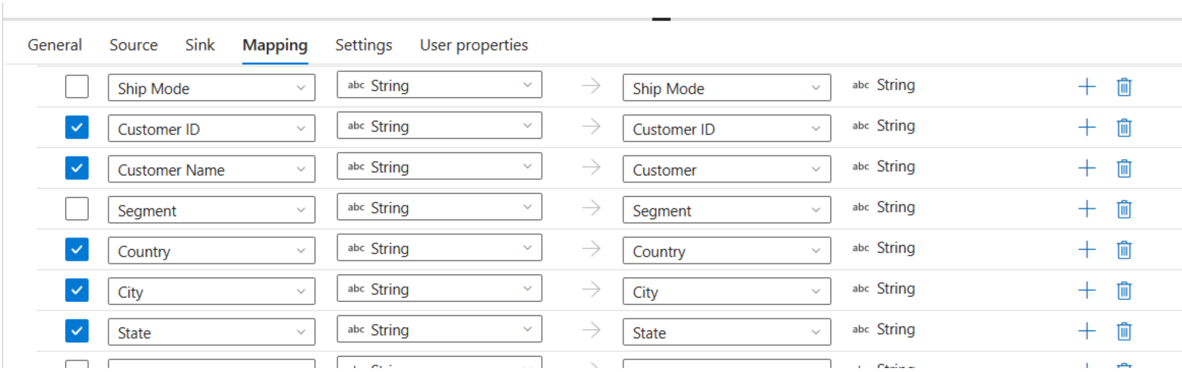

Figure 18 Column mapping: 'Customer Name' renamed to 'Customer' and selected columns.

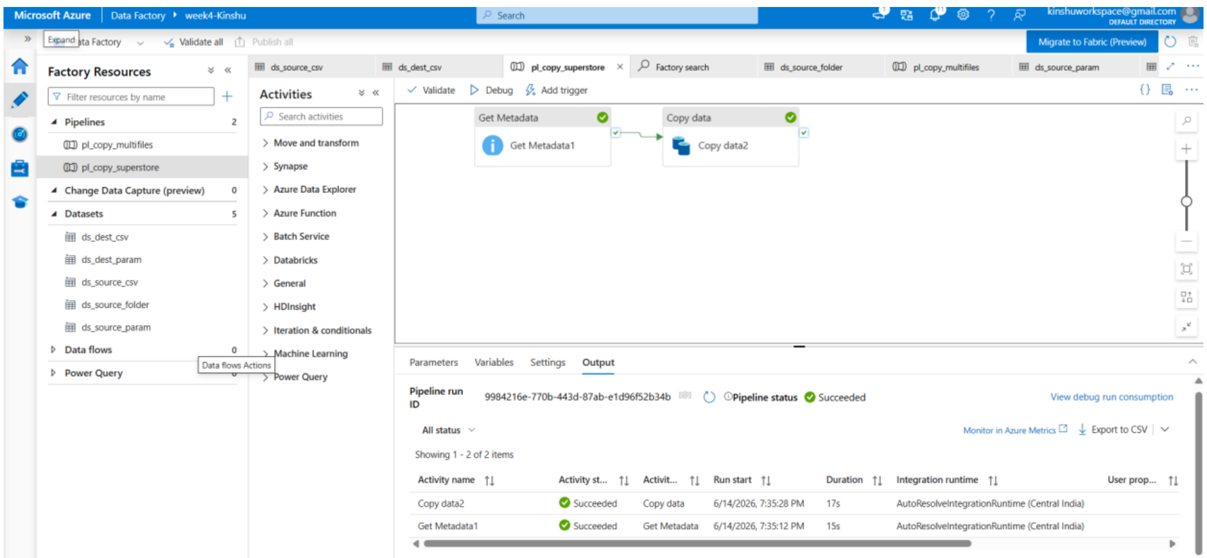

Figure 19 Transformation pipeline executed successfully with fault tolerance enabled.

Brief Summary

In this assignment, I created an Azure Resource Group and Azure Storage Account, created Blob containers, and then uploaded the Superstore CSV data set. An Azure Data Factory was deployed, along with Blob Storage Linked Service, along with source and destination DelimitedText datasets. This pipeline involves a Get Metadata Activity to check the source file properties including name, size, and column number, followed by a Copy Data Activity to copy the CSV from the source Blob Container to the destination Blob Container. The execution of this pipeline via Debug was successful, and the destination data was validated using metadata verification.

In terms of security, this was done through RBAC Reader for the user, and Storage Blob Data Contributor for the Azure Data Factory managed identity since the latter can access storage without using account keys. In addition to the minimum requirements for this assignment, I also developed a dynamic ForEach Pipeline for copying files from the source Blob Container to another Blob Container. Furthermore, this was used to demonstrate intransit data transformation using copy transformation for selecting columns and renaming.